# How a Windows eBPF service map is updated

Code walkthrough: reconciler → pkg/loadbalancer/maps → pkg/bpf → ebpf-go → eBPF-for-Windows

---

**Key idea:** pkg/loadbalancer/maps is fully platform-agnostic. All Windows behaviour lives in pkg/bpf's build-tagged files and is applied once at map creation; the per-entry Update path is identical to Linux.

## 1. Reconciler builds the entry and calls the LB-maps API

**pkg/loadbalancer/reconciler/bpf_reconciler.go:1266**

```
func (ops *BPFOps) upsertService(svcKey maps.ServiceKey, svcVal maps.ServiceValue) error {
    svcKey = svcKey.ToNetwork()   // host -> network byte order
    svcVal = svcVal.ToNetwork()
    err = ops.LBMaps.UpdateService(svcKey, svcVal)   // hand off to pkg/loadbalancer/maps
    ...
}
```

## 2. pkg/loadbalancer/maps dispatches v4/v6 to a generic *bpf.Map

**pkg/loadbalancer/maps/lbmaps.go:565 (and ctor at :153)**

```
func (r *BPFLBMaps) UpdateService(key ServiceKey, value ServiceValue) error {
    switch key.(type) {
    case *Service4Key:
        return r.service4Map.Update(key, value)   // -> pkg/bpf
    case *Service6Key:
        return r.service6Map.Update(key, value)
    }
}

// service4Map was built generically with the LINUX map type:
func NewService4Map(maxEntries int) *bpf.Map {
    return bpf.NewMap("cilium_lb4_services_v2", ebpf.Hash,
        &Service4Key{}, &Service4Value{}, maxEntries, ...)
}
```

## 3. pkg/bpf marshals + writes via ebpf-go

**pkg/bpf/map_impl.go:1335**

```
func (m *Map) Update(key MapKey, value MapValue) error {
    ...
    if err = m.open(); err != nil { return err }
    err = m.m.Update(key, value, ebpf.UpdateAny)   // ebpf-go -> efw runtime on Windows
    ...
}
// m.m is the ebpf-go *ebpf.Map. On Windows ebpf-go routes Update to eBPF-for-Windows.
// The write reaches the datapath's real object only because the map was bound to it
// at create time -- see the three Windows glue pieces below.
```

## The Windows glue — applied once, at OpenOrCreate

These build-tagged pieces (no-ops on Linux) bind the agent's map to the same object the efw datapath reads, and make the spec acceptable to efw.

## (a) Pin path → /ebpf/global/<name>

**pkg/bpf/bpffs_windows.go:26**

```
func agentMapPath(name string) string {
    return EbpfGlobalPinPrefix + "/" + name   // "/ebpf/global/" + name, forward slashes
}
```

## (b) Prefer the datapath's legacy pin

**pkg/bpf/map_impl.go:631 -> pkg/bpf/map_legacy_windows.go:57**

```
// map_impl.go, inside openOrCreate():
m.applyLegacyMapAlias()

// map_legacy_windows.go:
func (m *Map) applyLegacyMapAlias() {
    aliases, ok := legacyMapAliases[m.name]   // cilium_lb4_services_v2 -> cilium_lb4_services
    if !ok { return }
    for _, legacy := range aliases {
        legacyPath := MapPath(m.Logger, legacy)
        if !pinExists(legacyPath) { continue }
        m.name = legacy                        // reuse datapath's map (id 26)
        m.path = legacyPath
        if m.spec != nil { m.spec.Name = legacy }
        return
    }
}
```

## (c) Translate the map type/flags efw understands

**pkg/bpf/bpf_ebpf.go:21 -> pkg/bpf/maptype_windows.go:24**

```
// bpf_ebpf.go, inside createMap():
spec.Type  = ToPlatformMapType(spec.Type)    // ebpf.Hash -> ebpf.WindowsHash
spec.Flags = ToPlatformMapFlags(spec.Flags)  // drop Linux BPF_F_* (efw rejects them)
m, err := ebpf.NewMapWithOptions(spec, *opts)

// maptype_windows.go:
func ToPlatformMapType(t ebpf.MapType) ebpf.MapType {
    switch t {
    case ebpf.Hash:    return ebpf.WindowsHash
    case ebpf.LRUHash: return ebpf.WindowsLRUHash
    case ebpf.LPMTrie: return ebpf.WindowsLPMTrie
    ...
    }
}
func ToPlatformMapFlags(flags uint32) uint32 { return 0 }
```

## End-to-end

```
upsertService (reconciler)
  |- BPFLBMaps.UpdateService            pkg/loadbalancer/maps   (generic, no OS logic)
       |- (*bpf.Map).Update             pkg/bpf/map_impl.go
            |- ebpf-go m.m.Update(key,value,UpdateAny)  --> eBPF-for-Windows map

  bound once at OpenOrCreate via:
    agentMapPath          -> /ebpf/global/cilium_lb4_services
    applyLegacyMapAlias   -> reuse datapath's map (id 26)
    ToPlatformMapType/Flags -> WindowsHash, no BPF_F_* flags
```

The Service4Key / Service4Value structs (with ToNetwork() + MarshalBinary) are the exact 12-byte key / 12-byte value layouts seen in `bpftool map dump`; ebpf-go serializes them straight into the efw map.